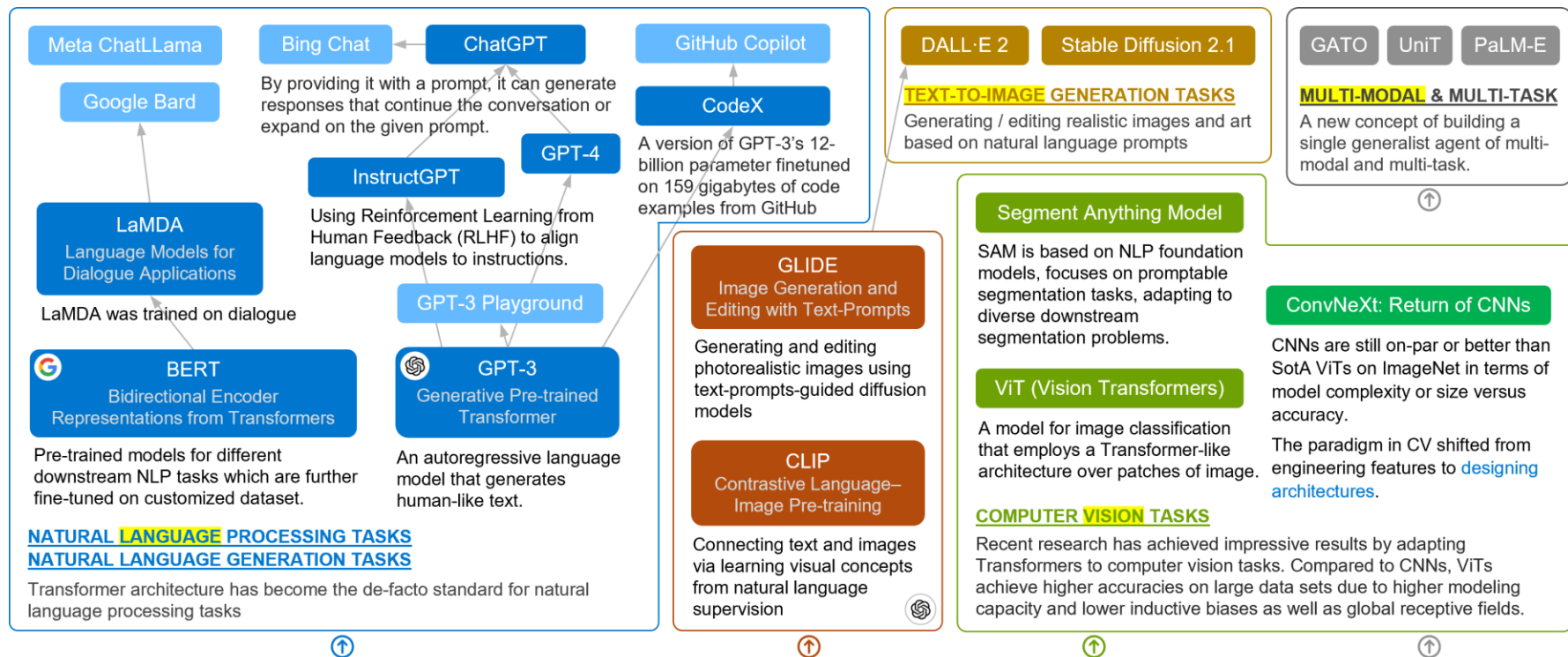


- ① Transformer is All You Need
- ② Large Language Models
- ③ Generative AI Techniques

Transformers Take Over Artificial Intelligence



Transformers Large-scale Transformer models triggered breakthrough in AI-Generated Content

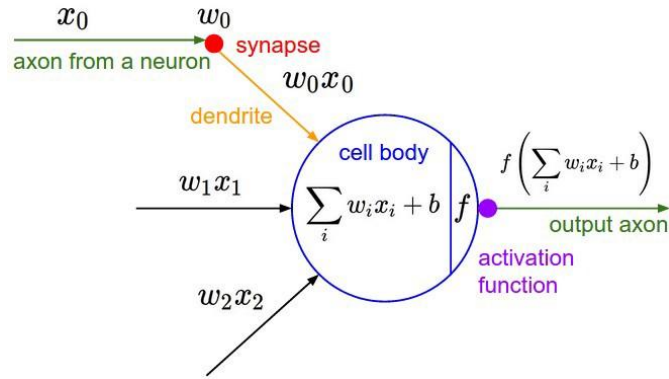
A Transformer is a type of neural network architecture based on the concept of self-attention, which allows the model to focus on specific parts of the input sentence to better understand the context of the sentence. Transformer models have been used to achieve state-of-the-art results in many NLP tasks, such as machine translation, text summarization, and question answering.

Transformer is All You Need

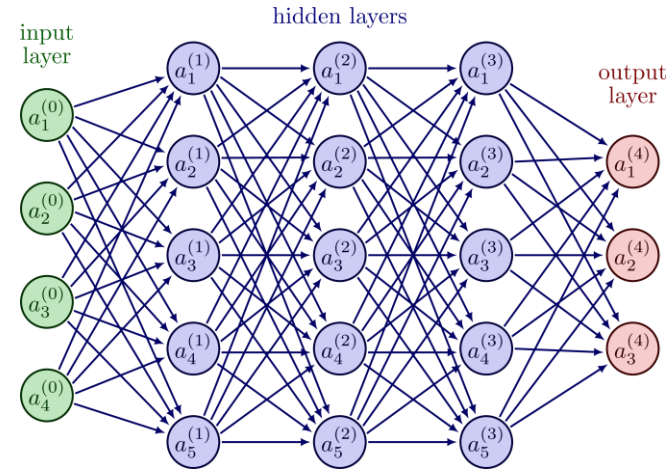
Sanping Li

Apr 2023

Neuron and Artificial Neural Networks



<https://cs231n.github.io/neural-networks-1/>

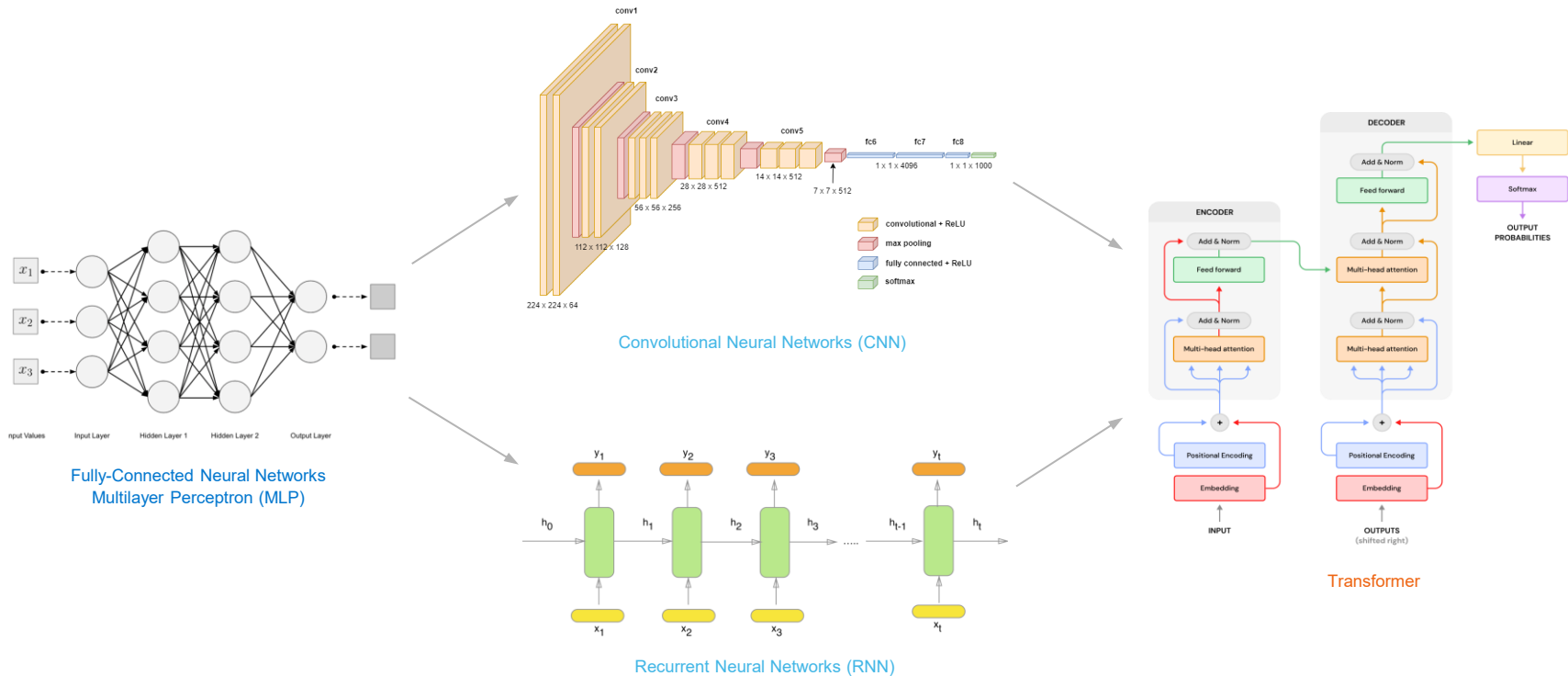


https://tikz.net/neural_networks/

The fundamental idea of neural networks is to create a flexible “computing fabric” out of a large number of simple (essentially identical) components — and to have this “fabric” be one that can be incrementally modified to learn from examples.

<https://writings.stephenwolfram.com/2023/02/what-is-chatgpt-doing-and-why-does-it-work/>

Evolution of Deep Learning Model Architectures

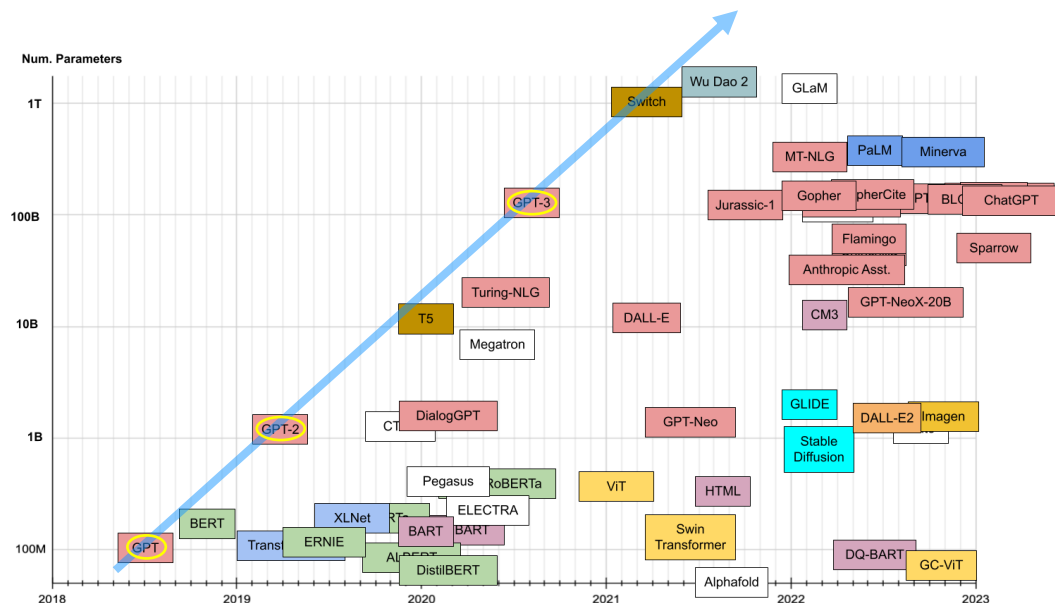


Transformer is Changing the Game

- Incredibly **large** models
 - Thanks to Transformers' computational efficiency and scalability, it has become possible to train deep learning models of unprecedented size.
- It is reinventing RNN and CNN domains.
 - With increasing practical applications, Transformers gain more confidence to upgrade RNN and fuse CNN.
 - The **unification of deep learning architectures** based on the Transformer model is a trend.
- Transformer itself is still **evolving**.
 - With the models and datasets growing, **there is still no sign of saturating performance**.

Size. Scale. More parameters.

- If there's one trend that has, above all others, dominated AI in the last five or six years, it is the **steady increase in parameter count of the best models**, which I've seen referred to as **Moore's law for large LMs**.
- The GPT family is the clearest — albeit not the only — embodiment of this fact.



Transformer Architecture

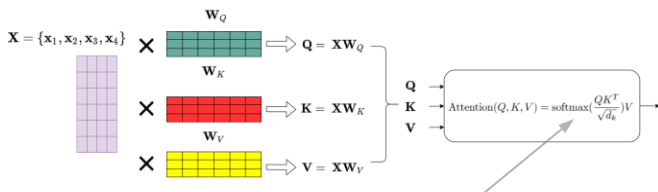
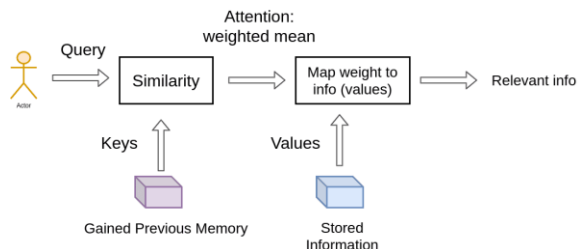
Transformer Architecture

- The transformer adopts an architecture based on [encoder-decoder stacking](#), with the use of [attention mechanism](#), differentially weighing the significance of each part of the input data.
- Each encoder consists of a [Self-Attention layer](#) followed by the [Feed Forward network](#). Self-attention uses trained [embeddings](#) from the same layer to compute the attention vector. It passes its encodings to the next encoder layer as inputs.
- The decoder also has the same layers as the encoder, except that [additional encoder-decoder attention](#) is introduced in between to help the model extract relevant features from attention vectors from the encoder.
- Both the encoder and decoder layers have a feed-forward neural network for additional processing of the outputs, and contain [residual connections and layer normalization](#) steps.



Attention is All You Need

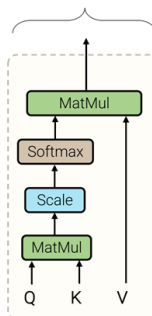
Following the database-query paradigm, this term simply finds the similarity of the searching query with an entry in a database.



Finally, we apply a softmax function to get the final **attention weights** as a probability distribution.

Computational and Memory Complexity

$$\mathcal{O}(n^2)$$

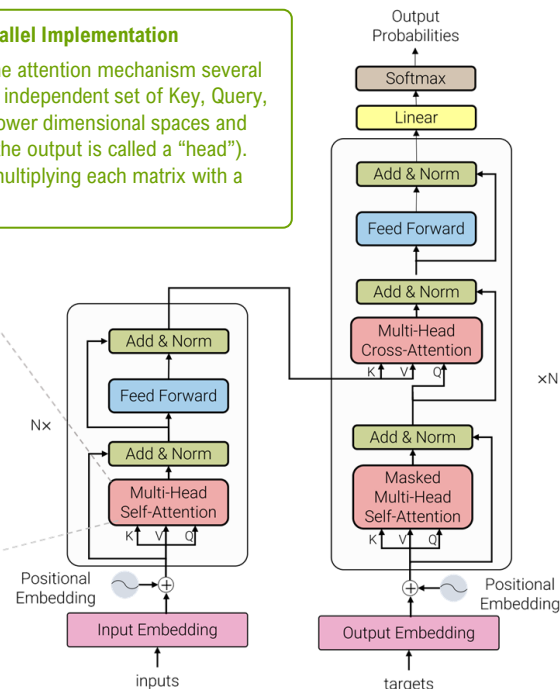


Multi-Head Attention and Parallel Implementation

In essence, we run through the attention mechanism several times. Each time, we map the independent set of Key, Query, Value matrices into different lower dimensional spaces and compute the attention there (the output is called a “head”). The mapping is achieved by multiplying each matrix with a separate weight matrix.

Self-attention

Self-attention, sometimes called intra-attention, is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence.



An Example of Attention Matrix

- Let's say the query is a sequence of **4 tokens** and the sequence that we would like to associate with, contains **5 tokens**.
- Both sequences contain vectors of the same embedding dimension, which is $d_{model} = 3$ in the example. Then the **self-attention** can be defined as **two matrix multiplications**.
- By **putting all the queries together**, we have a matrix multiplication instead of a single query vector to matrix multiplication every time.
- Each query is processed completely independently from the others**. This is the parallelization that we get for free by just using matrix multiplications and feeding all the input tokens/queries.

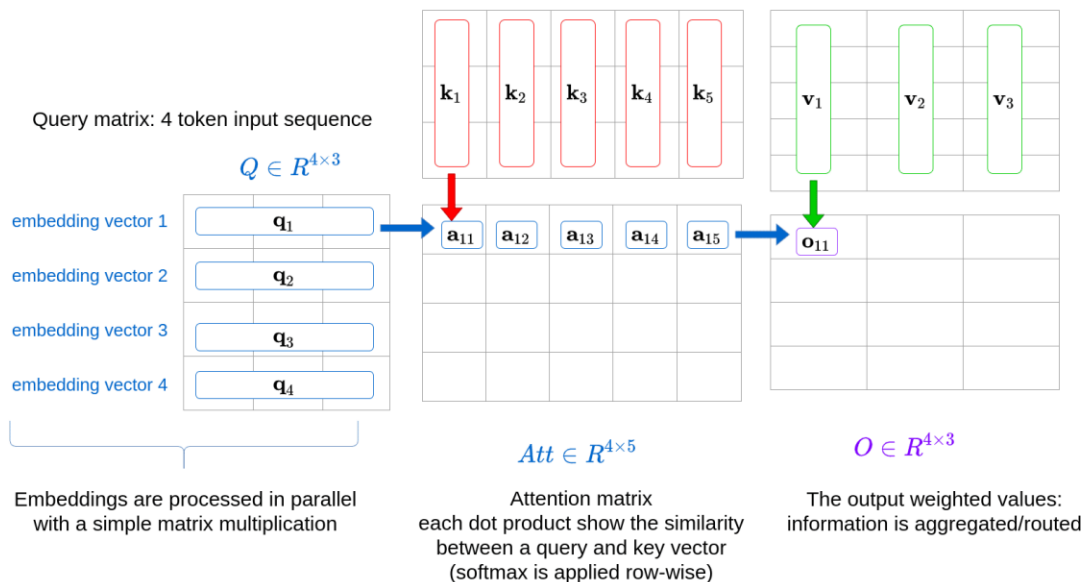
$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \mathbf{K} = \mathbf{X}\mathbf{W}^K, \mathbf{V} = \mathbf{X}\mathbf{W}^V, \mathcal{R}^{batch \times tokens \times d_k}$$

$$\mathbf{Y} = \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V}$$

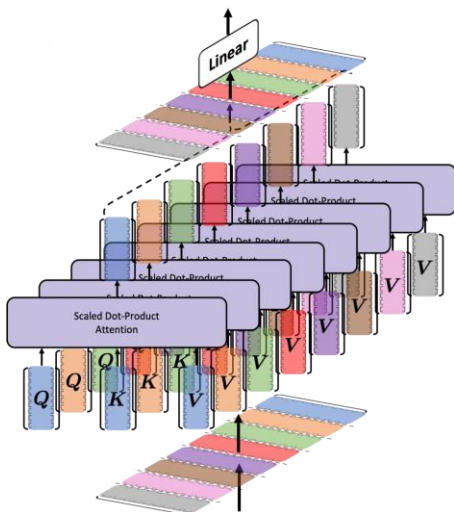
Keys, Values: 5 token sequence to associate with the input queries

$$\mathbf{K}^T \in \mathbb{R}^{3 \times 5}$$

$$\mathbf{V} \in \mathbb{R}^{5 \times 3}$$



Multi-Head Self-Attention Mechanism



```
# Let's see an example where the number of input sentence is 9 and
temp_mha = MultiHeadAttention(d_model=512, num_heads=8)
sample_sentence_source_lang = tf.random.uniform(1, 9, 512)) # (1, 9, 512)
sample_sentence_target_lang = tf.random.uniform(1, 12, 512)) # (1, 12, 512)
out, attn = temp_mha(v=sample_sentence_source_lang, k=sample_sentence_target_lang)

# In the results below, you can see that you reweight the "values"
# in each head, and the size of the resulting 'scaled_attention'
```

Inside 'MultiHeadAttention' class...

The shape of 'q' is (1, 12, 512)
The shape of 'k' is (1, 9, 512)
The shape of 'v' is (1, 9, 512)

After passing 'q', 'k', 'v' through densely connected layers...
The shape of 'q' is (1, 12, 512)
The shape of 'k' is (1, 9, 512)
The shape of 'v' is (1, 9, 512)

After splitting the heads...
The shape of 'q' is (1, 8, 12, 64)
The shape of 'k' is (1, 8, 9, 64)
The shape of 'v' is (1, 8, 9, 64)

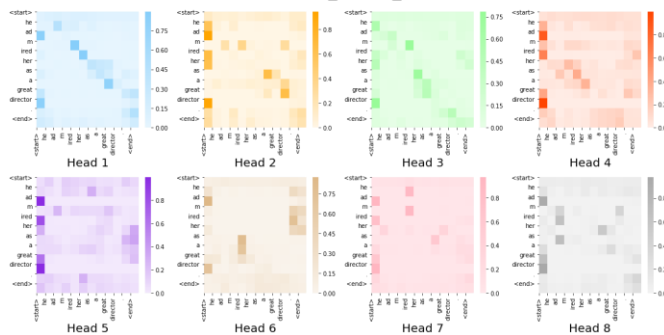
The shape of 'attention_weights' is (1, 8, 12, 9)
The shape of 'scaled_attention' is (1, 8, 12, 64)

After transposing...
The shape of 'scaled_attention' is (1, 12, 8, 64)

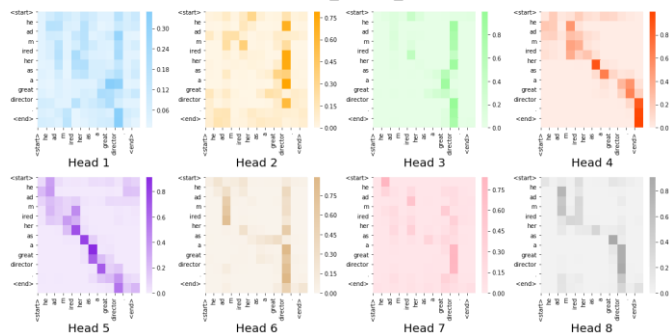
The shape of 'concat_attention' is (1, 12, 512)

The shape of 'output' is (1, 12, 512)

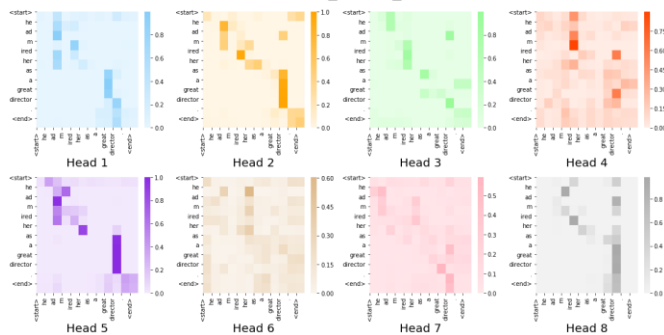
The heat maps of self attentions of "He admired her as a great director."
in encoder_layer1_block



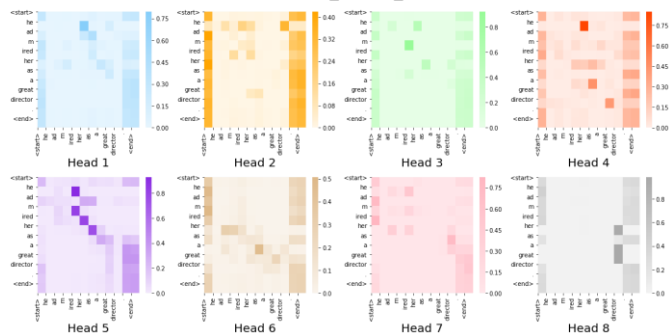
The heat maps of self attentions of "He admired her as a great director."
in encoder_layer2_block



The heat maps of self attentions of "He admired her as a great director."
in encoder_layer3_block



The heat maps of self attentions of "He admired her as a great director."
in encoder_layer4_block



Tokens

<https://docs.cohere.ai/docs/tokens>

- Language models understand "tokens" rather than characters or bytes.
 - Tokens are actually the building blocks of NLP and all the NLP models process raw text [at the token level](#).
- One token can be a part of a word, an entire word, or punctuation.
 - Very common words like "water" will have their own unique tokens. A longer, less frequent word might be encoded into 2-3 tokens, e.g. "waterfall" gets encoded into two tokens, one for "water" and one for "fall".
- Tokenization** is one of the most important steps in text pre-processing. Tokenization in simple words is the process of splitting a phrase, sentence, paragraph, one or multiple text documents into smaller units.
 - For example, A subword-based tokenization algorithm will break the sentence into subwords, like ["Let", "us", "learn", "token", "ization."]
 - Note that tokenization is sensitive to whitespace and capitalization.
- Tokens are used to form the **vocabulary**, which is a set of unique tokens in a corpus (a dataset in NLP). This vocabulary is then converted into **numbers** (IDs) and helps us in modeling.



"adays": 11113,	"zza": 26786,
"adaÅt": 16363,	"zzarella": 38808,
"add": 25224,	"zzle": 9260,
"adder": 29901,	"zso": 35130,
"addin": 41284,	"zÅuh": 23242,
"ade": 702,	"zÅuh": 8025,
"adace": 25978,	"zÅuh": 23876,
"aded": 12777,	"zÅl": 11852,
"adel": 16325,	"{"": 90,
"adelph": 26598,	"{\\"": 35000,
"adelphia": 28850,	" ": 91,
"adem": 49290,	" ": 92,
"aden": 14771,	" }": 29464,
"adequ": 34209,	"_": 93,
"ader": 8312,	"_["": 11394,
"aders": 15221,	"_["": 45361,
"ades": 2877,	"__": 47014,
"adesh": 26338,	"_}": 25010,
"adh": 46908,	"_?": 26911,
"adi": 5688,	"_}": 27456,
"adian": 42343,	"_="": 5990,
"adian": 42985,	"_="": 20409,
"adic": 43341,	"_="": 29026,
"adin": 39961,	" ": 94,
"ading": 8166,	" ": 6010,
"adium": 13534,	" ["": 42229,
"adile": 43212,	" ["": 5675,
"ado": 1573,	" ["": 1055,
"adomo": 40633,	" ["": 11737,
"ador": 5409,	" ["": 10332,
"adora": 23020,	" ["": 42087,
"adore": 40095,	" ": 95,
"adores": 11856,	"e["": 48506,
"adors": 29816,	"E": 96,
"ados": 4181,	"E": 13033,
"adow": 11528,	"E": 29693,
"ados": 33480,	"E["": 11372,
"ados": 23029,	"E["": 47412,
"adst": 5834,	"E["": 21305,
"adt": 14680,	"E["": 24037,
"adura": 25154,	"E": 97,

<https://towardsdatascience.com/word-subword-and-character-based-tokenization-know-the-difference-ea0976b64e17>

OpenAI's Tokenizer

GPT-3 Codex

If you need a programmatic interface for tokenizing text, check out the transformers package for python or the gpt-3-encoder package for node.js.

Clear

Show example

Tokens Characters

35

145

If you need a programmatic interface for tokenizing text, check out the transformers package for python or the gpt-3-encoder package for node.js.

TEXT TOKEN IDS

```
[1532, 345, 761, 257, 1430, 13849, 7071, 329, 11241, 2890, 2420, 11,
2198, 503, 262, 6121, 364, 5301, 329, 21015, 393, 262, 308, 457, 12, 18,
12, 12685, 12342, 5301, 329, 10139, 13, 8457, 13]
```

TEXT TOKEN IDS

GPT-3 Codex

```
pipe = MediaPipeFace(static_image_mode=True, max_num_faces=1,
refine_landmarks=True)
disp = DisplayFace(draw3d=True, refine_landmarks=True)
file = '../data/sample/mona.png'
else:
    print('Undefined mode only the following modes are available: \nface
/ hand / body / holistic')
    sys.exit()
```

Clear

Show example

Tokens Characters

173

518

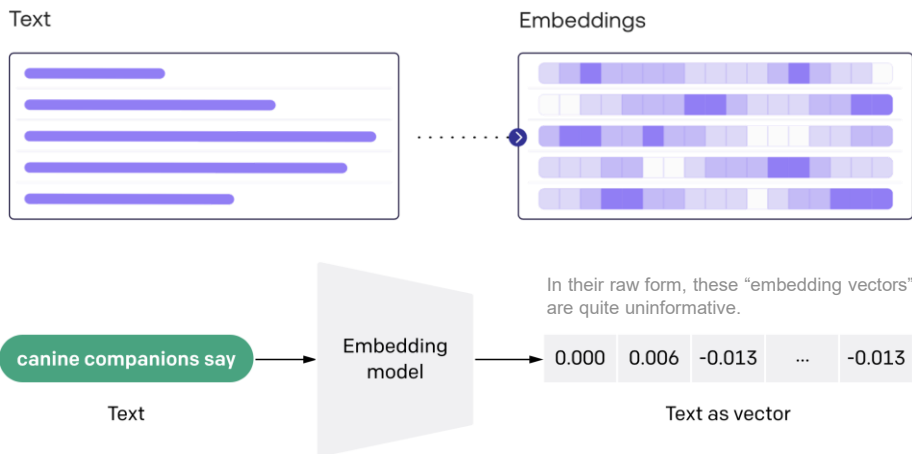
```
# Load mediapipe and display class
if mode=='face_detect':
    pipe = MediaPipeFaceDetect(model_selection=0, max_num_faces=5)
    disp = DisplayFaceDetect()
    file = '../data/sample/mona.png'
elif mode=='face':
    pipe = MediaPipeFace(static_image_mode=True, max_num_faces=1, refine_
landmarks=True)
    disp = DisplayFace(draw3d=True, refine_landmarks=True)
    file = '../data/sample/mona.png'
else:
    print('Undefined mode only the following modes are available: \nface
/ hand / body / holistic')
    sys.exit()
```

TEXT TOKEN IDS

<https://platform.openai.com/tokenizer>

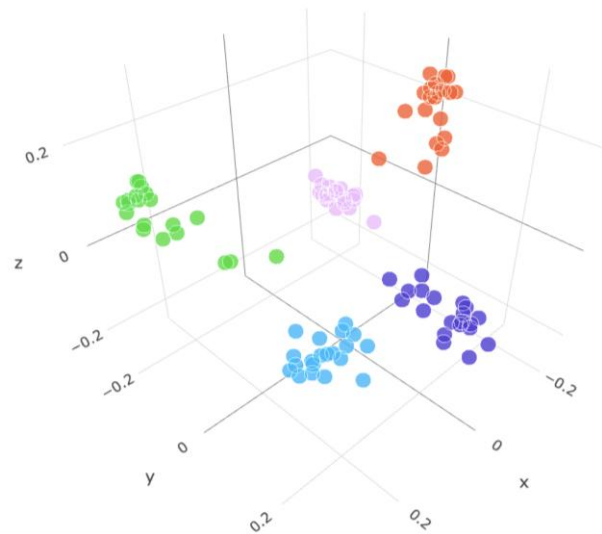
Embeddings

- Embeddings are a way to represent the meaning of text as [a list of numbers](#).
- This is useful because once text is in this form, it can be compared to other text for [similarity](#).
 - Using a simple comparison function, we can calculate a similarity score for two embeddings to figure out whether two texts are talking about similar things.



If we do things like measure distances between these vectors, then we can find things like “nearnesses” of words.

● animal ● athlete ● film ● transportation ● village



Embeddings from the `text-similarity-babbage-001` model, applied to the DBpedia dataset. We randomly selected 100 samples from the dataset covering 5 categories, and computed the embeddings via the `/embeddings` endpoint. The different categories show up as 5 clear clusters in the embedding space. To visualize the embedding space, we reduced the embedding dimensionality from 2048 to 3 using PCA. The code for how to visualize embedding space in 3D dimension is available here.

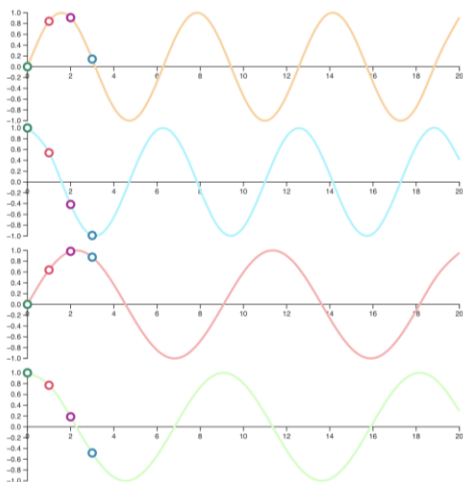
Positional Encoding in Transformer

Sequence	Index of token	Positional Encoding Matrix			
I	0	P_{00}	P_{01}	...	P_{0d}
am	1	P_{10}	P_{11}	...	P_{1d}
a	2	P_{20}	P_{21}	...	P_{2d}
Robot	3	P_{30}	P_{31}	...	P_{3d}

Positional Encoding Matrix for the sequence 'I am a robot'

Sequence	Index of token, k	Positional Encoding Matrix with $d=4, n_{\text{model}}$			
I	0	$P_{00}=\sin(0)=0$	$P_{01}=\cos(0)=1$	$P_{02}=\sin(0)=0$	$P_{03}=\cos(0)=1$
am	1	$P_{10}=\sin(1/1)=0.84$	$P_{11}=\cos(1/1)=0.54$	$P_{12}=\sin(1/10)=0.10$	$P_{13}=\cos(1/10)=1.0$
a	2	$P_{20}=\sin(2/1)=0.91$	$P_{21}=\cos(2/1)=-0.42$	$P_{22}=\sin(2/10)=0.20$	$P_{23}=\cos(2/10)=0.98$
Robot	3	$P_{30}=\sin(3/1)=0.14$	$P_{31}=\cos(3/1)=-0.99$	$P_{32}=\sin(3/10)=0.30$	$P_{33}=\cos(3/10)=0.96$

Positional Encoding Matrix for the sequence 'I am a robot'



p_0	p_1	p_2	p_3	
0.000	0.841	0.909	0.141	$i=0$
1.000	0.540	-0.416	-0.990	$i=1$
0.000	0.638	0.983	0.875	$i=2$
1.000	0.770	0.186	-0.484	$i=3$

Positional Encoding

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

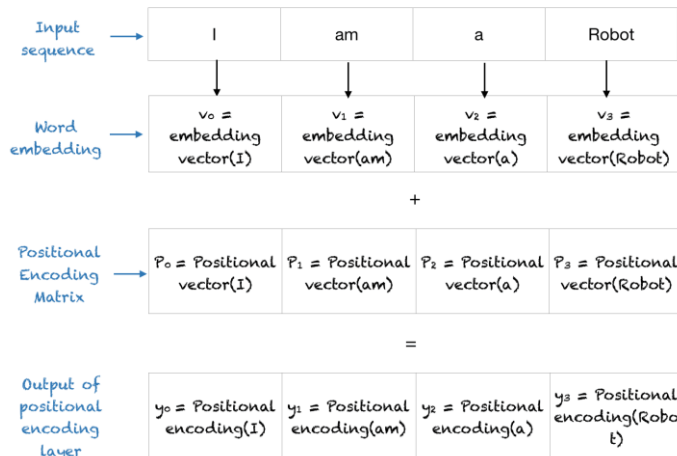
$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Settings: $d = 50$

The value of each positional encoding depends on the position (pos) and dimension (d). We calculate result for every index (i) to get the whole vector.

As you can see these values depend on d_{model} (input dimension) and i (index of the position vector).

Why does one just add the token-value and token-position embedding vectors together? I don't think there's any particular science to this. It's just that various different things have been tried, and this is one that seems to work.

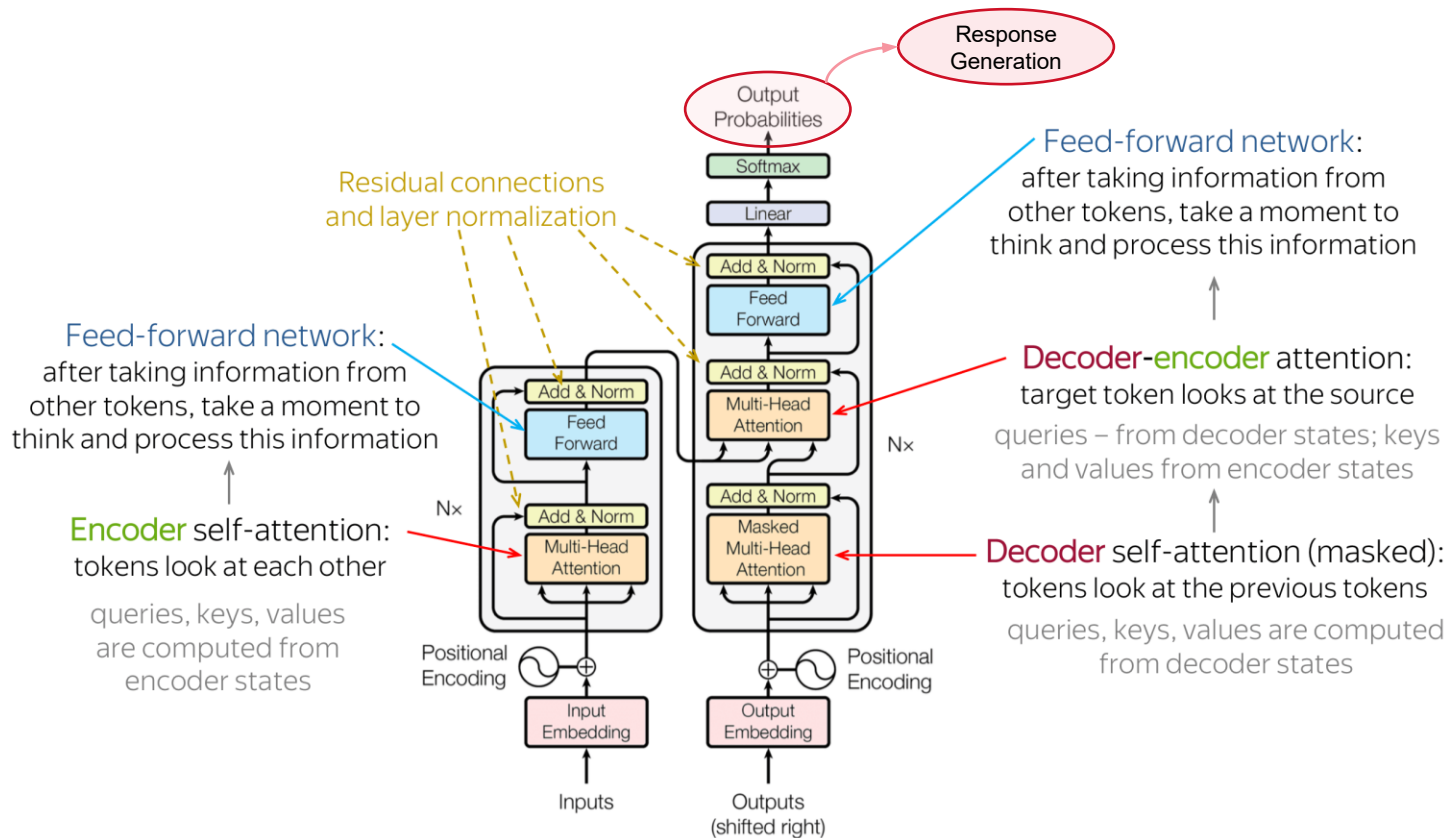


- In languages, the order of the words and their position in a sentence really matters. The transformer model, however, does not use recurrence or convolution and treats each data point as independent of the other.
- Hence, positional information is added to the model explicitly to retain the information regarding the order of words in a sentence.
- Positional encoding describes the location or position of an entity in a sequence so that each position is assigned a unique representation.
- Transformers use a smart positional encoding scheme, where each position/index is mapped to a vector. Hence, the output of the positional encoding layer is a matrix, where each row of the matrix represents an encoded object of the sequence summed with its positional information.

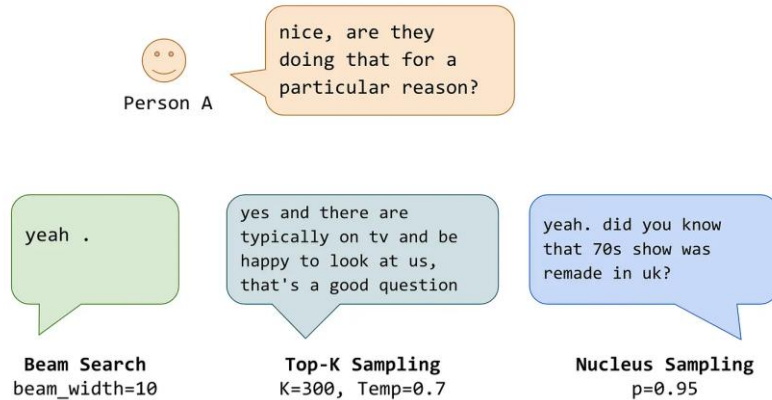
<https://machinelearningmastery.com/a-gentle-introduction-to-positional-encoding-in-transformer-models-part-1/>

<https://erdem.pl/2021/05/understanding-positional-encoding-in-transformers>

Put It All Together



Diversity of Outputs Generated



<https://towardsdatascience.com/decoding-strategies-that-you-need-to-know-for-response-generation-ba95ee0faadc>

- Language models generate some text as their output.
- There are a couple of commonly used algorithms used by all of these applications as part of their last step to produce their final output.
- **Greedy Search** is one such algorithm. It is used often because it is simple and quick.

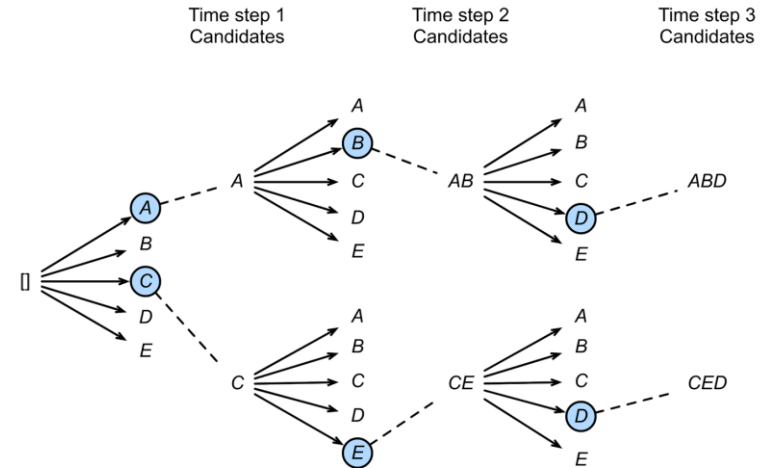


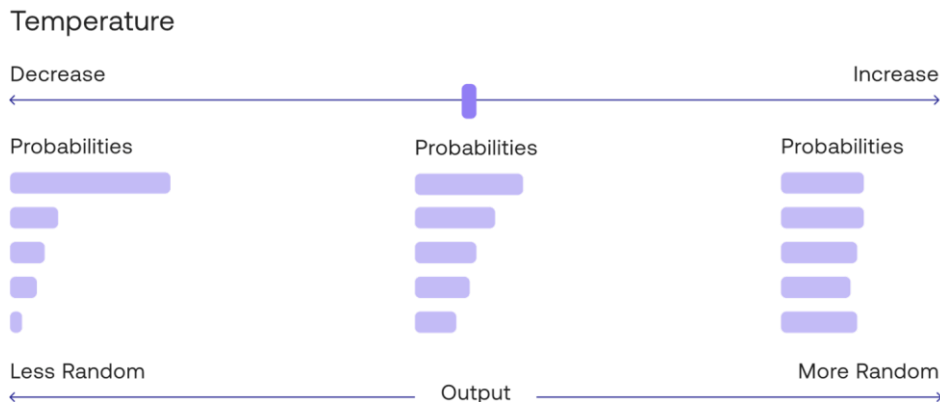
Fig. 10.8.3 The process of beam search (beam size: 2, maximum length of an output sequence: 3). The candidate output sequences are *A*, *C*, *AB*, *CE*, *ABD*, and *CED*

https://d2l.ai/chapter_recurrent-modern/beam-search.html

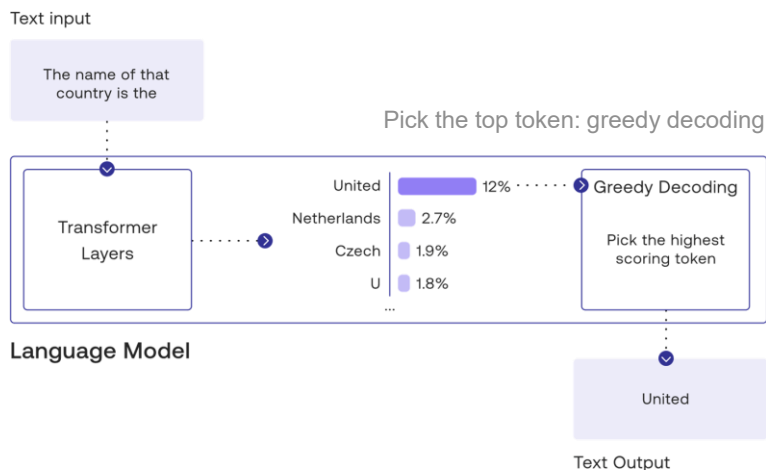
- The alternative is to use **Beam Search**. It is very popular because, although it requires more computation, it usually produces much better results.
- At each timestep, it generates all possible tokens in the vocabulary list; then, it will choose top B candidates that have the most probability. Those B candidates will move to the next time step, and the process repeats. In the end, there will only be B candidates.

Temperature

- Sampling from generation models incorporates randomness, so that the same prompt may yield different outputs each time you hit "generate". Temperature is a number used to tune **the degree of randomness** (Usually, the range is $0 < \text{temp} \leq 1$).
- **A lower temperature means less randomness**; a temperature of 1 will always yield the same output. Lower temperatures (less than 1) are more appropriate when performing tasks that have a "correct" answer, like question answering or summarization.
 - If the model starts repeating itself this is a sign that the temperature is too low.
- **High temperature means more randomness**, which can help the model give more creative outputs.
 - If the model starts going off topic or giving non-sensical outputs, this is a sign that the temperature is too high.
- As sequences get longer, the model naturally becomes more confident in its predictions, so you can raise the temperature much higher for long prompts without going off topic. In contrast, using high temperatures on short prompts can lead to outputs being very unstable.

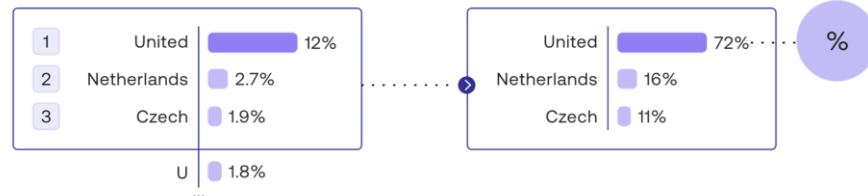


Decoding Strategies: Picking the Output Token



1) Consider only the top 3 tokens.
Ignore all others.

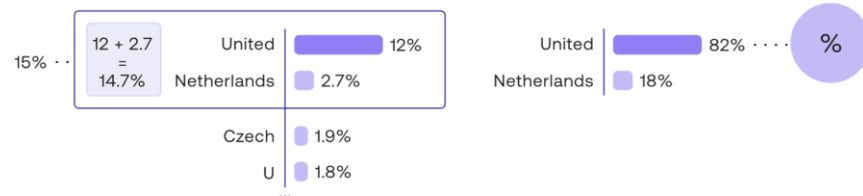
2) Sample from them based on
their likelihood scores.



Pick from amongst the top tokens: **top-k**. This approach allows the other high-scoring tokens a chance of being picked. The randomness introduced by this sampling helps the quality of generation in a lot of scenarios.

1- Consider only the top tokens whose
likelihoods add up to 15%. Ignore all others.

2- Sample from them based on their
likelihood scores.



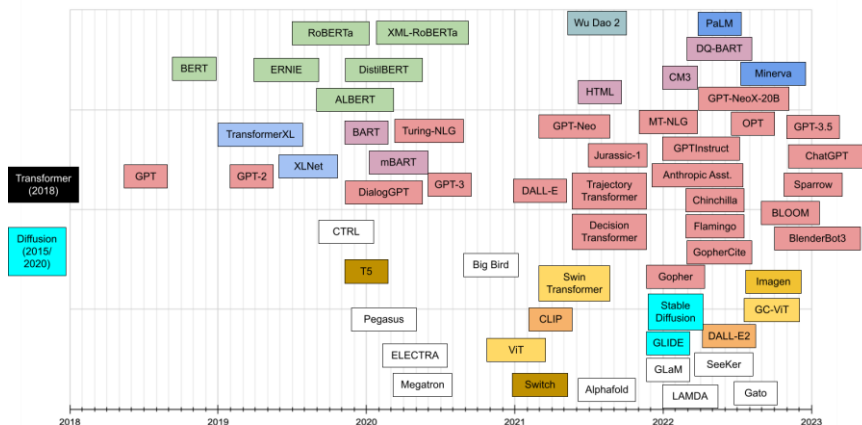
This method, called **Nucleus Sampling**, shortlists the top tokens whose sum of likelihoods does not exceed a certain value. A toy example with a **top-p** value of 0.15.

Family of Transformer Models

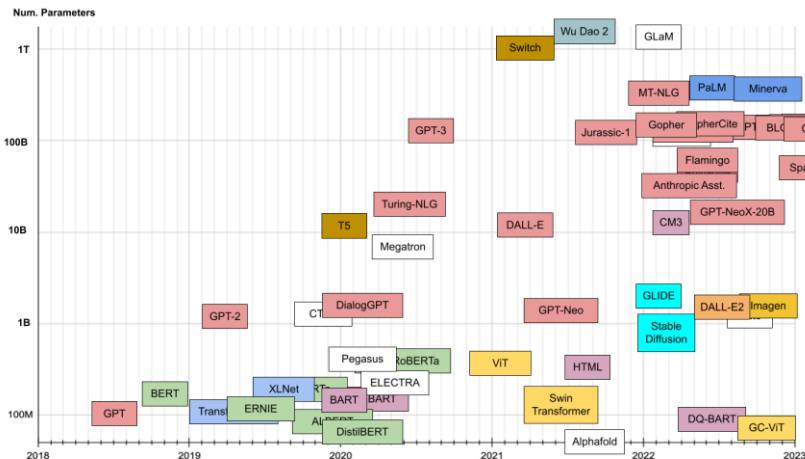
Family of Transformer Models

For the past few years, the size of large language models has **increased by a factor of ten per year**. It's beginning to resemble another Moore's Law.

Timeline of Transformer Models



Model size in millions of parameters



- Over the last five years, the Transformer architecture has become a popular choice for machine learning researchers, resulting in a proliferation of models with ever-growing model size in the number of parameters.
- As researchers have continued to explore the potential of Transformer-based models including applications with text, images, videos, and even multimodal data, the models utilizing this architecture have become increasingly complex and computationally intensive.

